



O que são Triggers no SQL e como funcionam?

Automatizando tarefas no seu banco de dados sem complicação — entenda como os gatilhos trabalham nos bastidores para manter seus dados sempre consistentes.

SQL PARA INICIANTES

BANCO DE DADOS

A Analogia do Mundo Real

Pense num robô automático 🤖

Um **Trigger** é como um assistente invisível que fica de olho nas mudanças do banco de dados e age imediatamente quando algo acontece — sem que você precise pedir.

O exemplo do carrinho de compras 🛒

Toda vez que você adiciona um produto ao carrinho em uma loja online, o valor total precisa ser recalculado automaticamente. O Trigger faz exatamente esse papel dentro do banco de dados, atualizando o total sem nenhuma intervenção manual!

- 📌 Triggers são **invisíveis para o usuário final**, mas essenciais para manter os dados corretos e atualizados em tempo real.



O Código na Prática

Veja abaixo o Trigger completo que recalcula o valor total de um pedido sempre que um novo item é inserido:

```
CREATE TRIGGER trg_atualiza_valor_pedido_insert
AFTER INSERT ON pedido_produto
BEGIN
    UPDATE pedido
    SET valor_pedido = (
        SELECT COALESCE(SUM(quantidade * preco_venda), 0)
        FROM pedido_produto
        WHERE id_pedido = NEW.id_pedido
    )
    WHERE id = NEW.id_pedido;
END;
```

Nas próximas seções, vamos dissecar cada parte desse código e entender o que cada linha faz. Não se preocupe — é mais simples do que parece!

Desmistificando o Código

Parte 1 — A Ativação do Gatilho

CREATE TRIGGER **trg_atualiza_valor_pedido_insert**

Registra e nomeia o "robô" no banco de dados. O nome é como uma identidade — deve ser único e descritivo para facilitar a manutenção futura.

AFTER INSERT ON **pedido_produto**

Define o **gatilho**: o Trigger "acorda" *depois* que um novo registro é inserido na tabela `pedido_produto`. O momento **AFTER** garante que o dado já foi salvo antes de agir.

BEGIN

Marca o início do bloco de instruções automáticas. Tudo que vem depois de **BEGIN** e antes de **END** é executado pelo Trigger.

Desmistificando o Código

Parte 2 — A Ação Principal

Atualizando a tabela de pedidos

```
UPDATE pedido  
SET valor_pedido = ( ... )
```

Indica qual tabela (`pedido`) e qual campo (`valor_pedido`) receberão o novo valor calculado. É aqui que o resultado aparece para o sistema.

A matemática por trás da mágica ✨

```
SELECT COALESCE(  
  SUM(quantidade * preco_venda),  
  0  
)
```

Multiplica a quantidade de cada item pelo seu preço, soma todos os valores e usa `COALESCE` para retornar **zero** caso o carrinho esteja vazio — evitando erros com valores nulos.

Desmistificando o Código

Parte 3 — A Precisão e o Encerramento

1 `WHERE id_pedido = NEW.id_pedido`

Filtra a soma para considerar **apenas os itens do pedido atual**. A palavra-chave `NEW` representa o registro que acabou de ser inserido — é como dizer "o pedido desta operação específica".

2 `WHERE id = NEW.id_pedido`

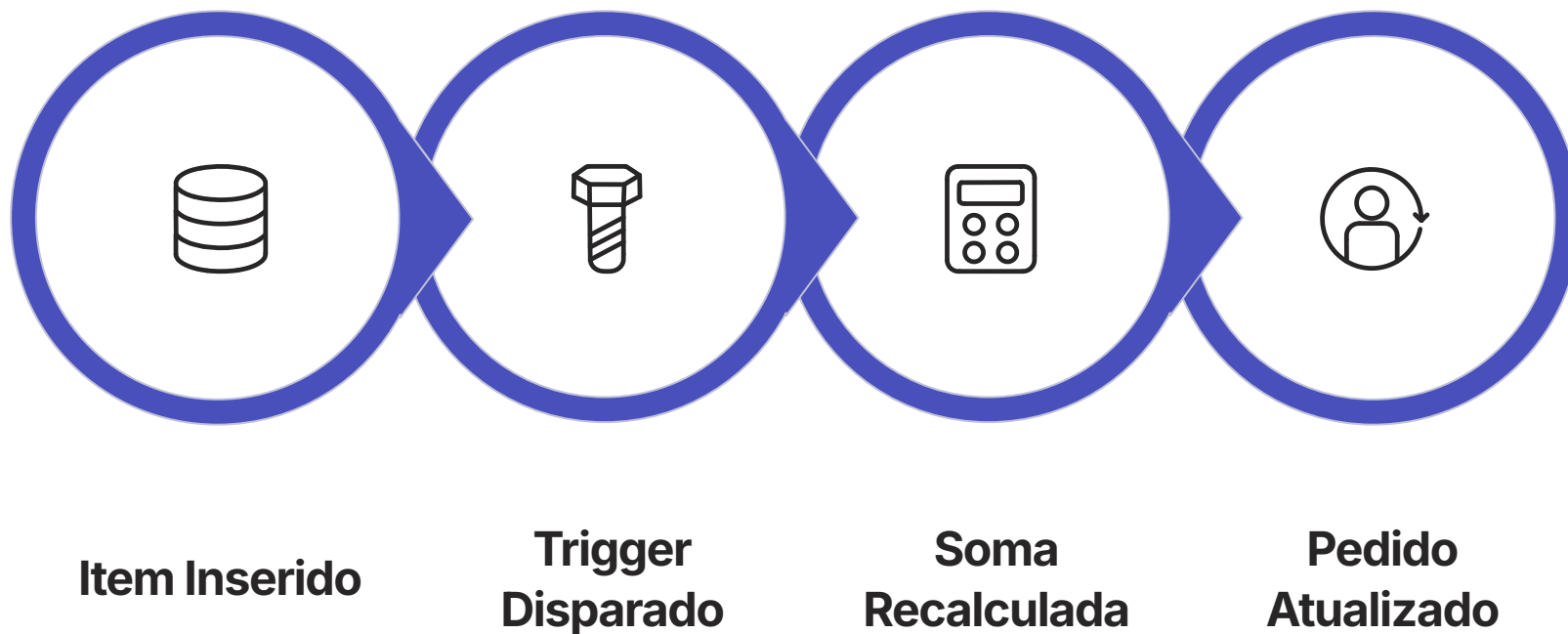
Garante que o valor calculado seja gravado **somente no pedido correto**, evitando que outros pedidos sejam acidentalmente alterados.

3 `END;`

Encerra o bloco de instruções do Trigger. O ponto e vírgula finaliza a declaração completa. O robô volta a hibernar até o próximo evento!

Resumo Visual do Fluxo

Veja como o Trigger funciona de ponta a ponta, do momento em que o item entra no banco até o valor do pedido ser atualizado:



Todo esse processo acontece em **milissegundos**, de forma transparente, garantindo que o valor do pedido esteja sempre correto sem esforço manual.

Outros Casos de Uso para Triggers



Auditoria de Dados

Registrar automaticamente quem alterou um registro, quando e o que foi mudado — criando um histórico completo de mudanças.



Validação e Segurança

Impedir que dados inválidos sejam inseridos ou bloquear alterações não autorizadas em tabelas críticas do sistema.



Notificações Automáticas

Disparar alertas ou atualizações em outros sistemas sempre que um evento específico ocorrer no banco de dados.



Sincronização de Tabelas

Manter tabelas espelho ou de backup atualizadas automaticamente quando os dados originais forem modificados.



Boas Práticas com Triggers

✓ O que fazer

- Nomeie seus Triggers de forma descritiva e padronizada
- Mantenha a lógica do Trigger simples e focada em uma tarefa
- Documente o propósito de cada Trigger com comentários no código
- Teste com dados reais antes de colocar em produção

⚠ O que evitar

- Evite criar Triggers em cascata que disparam outros Triggers
- Não coloque lógica de negócio complexa dentro de um único Trigger
- Cuidado com loops infinitos causados por atualizações circulares
- Não esqueça de testar o comportamento em casos extremos (carrinho vazio, etc.)



Recapitulando o que Aprendemos



O que é um Trigger

Um robô automático no banco de dados que reage a eventos como INSERT, UPDATE ou DELETE.



Exemplo real

Recalcular o total do carrinho toda vez que um produto é adicionado ao pedido.



Palavra-chave NEW

Representa o dado recém-inserido, permitindo filtrar e agir apenas sobre o registro correto.



Próximo passo

Experimente criar seu próprio Trigger! Comece simples: registre em uma tabela de log toda vez que um produto for atualizado.



Parabéns! Você agora entende como os Triggers funcionam e está pronto para aplicar esse conhecimento em projetos reais. 🎉